B M S COLLEGE OF ENGINEERING

(An Autonomous Institution Affiliated to VTU, Belagavi)

Post Box No.: 1908, Bull Temple Road, Bengaluru – 560 019

DEPARTMENT OF MACHINE LEARNING

Academic Year: 2026-2027 (Session: Feb 2026-July 2026)

INTRODUCTION TO ARTIFICIAL INTELLIGENCE

24AM4PCIAI

ALTERNATIVE ASSESSMENT TOOL (AAT)

Submitted by

Student Name:	NITYA R PAWAR
USN:	1BM24AI107
Semester & Section:	IV SEM & F

Valuation Report (to be filled by the faculty)

Score:		**Faculty In-charge:**	Prof. Soniya L
Comments:		Faculty Signature: with date	

Table of Contents

1. Abstract

The Tango puzzle is a logic-based constraint satisfaction problem in which a grid must be filled with Sun and Moon symbols such that every row and column contains an equal number of each symbol and no three identical symbols appear consecutively in any row or column. This project presents a Goal-Based Intelligent Agent that solves the Tango puzzle using the A* search algorithm. The agent represents the board as a two-dimensional grid, where each cell holds either a Sun symbol, a Moon symbol, or remains empty.

The heuristic function used in this implementation is h(n) = number of empty cells remaining, which guides the search toward states that are closer to a complete solution. The A* algorithm combines the actual cost g(n) with this heuristic estimate h(n) to evaluate and prioritise states for exploration. States that violate the puzzle constraints are discarded immediately, reducing unnecessary computation.

A Tkinter-based graphical user interface visualises the solving process and displays live telemetry including states explored, backtracking count, and elapsed time. The solver runs on a worker thread while the GUI remains responsive through a thread-safe event queue. The project demonstrates key artificial intelligence concepts including goal-based agents, PEAS formulation, state-space search, constraint satisfaction, and heuristic-guided search.

2. Introduction

2.1 Background

Artificial Intelligence is a field of computer science that focuses on creating systems capable of performing tasks that normally require human intelligence. Intelligent agents are systems that perceive their environment, process information, and take actions to achieve specific objectives. Search algorithms play an important role in artificial intelligence because they help agents explore possible solutions and select optimal paths. Puzzle-solving applications are widely used to demonstrate AI techniques because they require reasoning, state-space exploration, and decision making.

2.2 Problem Overview

Tango is a logic-based puzzle that requires users to fill a grid using two symbols: Sun and Moon. The puzzle follows predefined constraints which guide the solution process. The rules used in this project are:

- An equal number of Sun and Moon symbols must exist in every row and column.
- More than two identical symbols cannot appear consecutively.
- Empty cells must be filled while maintaining all constraints.

The problem can be modelled as a search problem where the intelligent agent explores multiple states to reach a valid completed solution.

2.3 Significance

The significance of this project includes:

- Demonstrates practical application of intelligent agents
- Shows use of search algorithms in problem solving
- Provides understanding of state-space representation

- Helps improve logical reasoning and decision making
- Demonstrates heuristic-based search techniques

3. Problem Description and Requirements

3.1 Problem Description

The objective of this project is to develop an intelligent agent capable of solving a Tango-based logic puzzle automatically using Artificial Intelligence techniques. The puzzle consists of a grid containing predefined Sun and Moon symbols along with several empty cells that need to be filled. The intelligent agent must determine appropriate symbols for the empty cells while ensuring that all puzzle constraints are satisfied.

The Tango puzzle can be represented as a search problem where each configuration of the puzzle forms a state in the state space. The agent begins with an initial partially completed puzzle and explores possible future states by placing valid symbols into empty positions. During this process, each generated state is checked against predefined rules to ensure correctness.

The major constraints considered in this project include maintaining an equal number of Sun and Moon symbols in each row and column and ensuring that more than two identical symbols do not appear consecutively. States violating these constraints are discarded immediately and are not considered for further exploration.

The system accepts the puzzle grid as input and generates a completed valid puzzle as output. In addition to solving the puzzle, the system also records performance-related information such as execution time, explored states, and backtracking statistics. The proposed implementation uses the A* search algorithm with a heuristic function to reduce unnecessary search operations and improve problem-solving efficiency.

Table 1 — Environment Characteristics

Property	Description
Observability	Fully Observable
Deterministic	Yes
Static	Yes
Discrete	Yes
Dynamic	No
Agent Type	Single Agent

3.2 Input and Output Requirements

Inputs:

- Puzzle grid size
- Initial symbols (Sun and Moon pre-placed cells)
- Empty cells to be filled by the agent

Outputs:

- Completed puzzle grid
- Performance statistics:
  - States explored
  - Execution time
  - Backtracking count

3.3 Functional Requirements

Table 2 — Functional Requirements

Requirement ID	Functional Requirement
FR1	Accept puzzle input
FR2	Generate neighbouring states
FR3	Validate puzzle constraints
FR4	Apply heuristic function
FR5	Execute A* search
FR6	Display puzzle solution
FR7	Generate performance statistics

The system should:

- Accept puzzle input
- Generate neighbouring states
- Validate constraints
- Apply heuristic evaluation
- Solve puzzle using A* algorithm
- Display solution visually

3.4 Non-Functional Requirements

Table 3 — Non-Functional Requirements

Requirement ID	Non-Functional Requirement
NFR1	Efficient execution time
NFR2	Accurate solution generation
NFR3	User-friendly visualisation
NFR4	Reliability
NFR5	Reduced memory usage

The system should:

- Produce accurate results
- Reduce unnecessary state exploration

- Provide user-friendly visualisation
- Maintain efficient execution time

4. Agent Design

4.1 Type of Agent

The proposed system uses a **Goal-Based Intelligent Agent** for solving the Tango-based logic puzzle. A goal-based agent is an intelligent system that selects actions based on a predefined objective. Unlike simple reflex agents, which make decisions only based on the current input, a goal-based agent considers the desired outcome before performing an action.

In this project, the objective of the intelligent agent is to obtain a fully completed puzzle grid while satisfying all predefined constraints. The agent continuously analyses the current state of the puzzle, generates possible future states, and evaluates them using the A* algorithm to determine which action should be taken next. Since the puzzle has a clearly defined target state, the goal-based approach is suitable for efficiently guiding the search process.

The agent repeatedly explores neighbouring states and moves toward the state that appears closer to the final solution. This decision-making process enables the agent to solve the puzzle efficiently while avoiding unnecessary exploration of invalid states.

4.2 Environment and PEAS Description

The agent operates on an N×N Tango grid. Using the PEAS framework, the environment is discrete, deterministic, fully observable, static, and single-agent. Table 4 gives the full PEAS specification.

Table 4 — PEAS Specification of the Tango Agent

Element	Description
Performance	Fill the Tango grid so every row and column has equal Suns and Moons, with no three identical symbols adjacent.
Environment	N×N Tango board — discrete, deterministic, fully observable, static, single-agent.
Actuators	place_symbol(row, col, symbol), remove_symbol(row, col).
Sensors	Current partial grid state — positions of all symbols placed so far.

4.3 State Space Representation

State-space representation is an important concept in artificial intelligence because it defines how a problem can be modelled and explored by an intelligent agent. In this project, each state represents the current configuration of the Tango puzzle grid, stored as a two-dimensional list where each cell holds SUN, MOON, or EMPTY.

The initial state consists of a partially filled grid containing predefined symbols and several empty cells. As the solving process progresses, the agent generates new states by filling empty cells with either Sun or Moon symbols while checking whether the generated state satisfies all puzzle constraints.

During the search process, the intelligent agent explores multiple such states and evaluates them using the A* algorithm and heuristic function. States violating the constraints are discarded immediately and

are not considered for further exploration. The goal state is reached when all empty cells are filled and all puzzle rules are satisfied successfully.

4.4 Agent Architecture

The system is built from two cooperating parts. The solver class, TangoAgent, contains all the search logic and does not import any interface library; it reports progress only through a callback function. The graphical application, TangoGUI, runs on the main thread, starts the solver on a separate worker thread and reads the events the solver produces in order to animate the board. The two threads communicate through a thread-safe queue, which keeps the interface responsive even while a long search is running.

4.5 Decision-Making Logic

At each step the agent uses the Minimum Remaining Values (MRV) heuristic to select the empty cell with the fewest currently valid symbol choices. It then tries each symbol in turn. If a symbol produces a valid partial state, the agent places it and recurses deeper; if not, the candidate is counted as a backtrack and the next symbol is tried. When no symbol is valid for the selected cell, the agent backtracks to the most recent valid placement and continues. The search ends successfully when no empty cells remain and all constraints are satisfied.

5. Algorithm and Implementation Details

This section describes the implementation in the order in which it executes: first the agent is created and its state initialised, then constraints are checked, then the recursive A*-guided search runs and finally the results are displayed.

5.1 A Search Algorithm*

The A* search algorithm is one of the most widely used informed search techniques in artificial intelligence. It combines actual path cost and estimated future cost to determine the most promising state for exploration. Unlike uninformed search methods that blindly explore possible states, A* uses heuristic information to guide the search toward the goal more efficiently.

The A* algorithm evaluates each state using the following evaluation function:

f(n) = g(n) + h(n)

Where:

- **g(n)** represents the actual cost from the initial state to the current state (number of cells already filled).
- **h(n)** represents the estimated remaining cost required to reach the goal state.

The state with the minimum evaluation value f(n) is selected for expansion. This process continues repeatedly until the goal state is reached. The use of the A* algorithm reduces unnecessary exploration and improves efficiency compared to uninformed search techniques.

5.2 Heuristic Function

A heuristic function provides an estimate of how close the current state is to the goal state. The effectiveness of the A* algorithm depends heavily on selecting an appropriate heuristic because it guides the search process.

In this project, the heuristic function is defined as:

h(n) = Number of empty cells remaining

This heuristic works by counting the number of unfilled cells in the puzzle grid. States containing a larger number of empty cells are considered farther from the solution, whereas states with fewer empty cells are considered closer to the goal state.

For example, if a puzzle state contains six empty cells, the heuristic value is six. After filling one valid cell, the heuristic value decreases to five. As the puzzle gradually approaches completion, the heuristic continues decreasing until it reaches zero at the goal state. This heuristic helps the algorithm prioritise states that appear closer to a completed solution.

5.3 Creating the Agent and Initialising State

The constructor stores the grid dimensions, creates a deep copy of the initial board, records the callback used to report events and an optional stop signal, and prepares the counters that track the search (explored states, backtracks, assignments, and maximum depth reached).

5.4 Constraint Validation

The method is_valid checks whether the current grid configuration satisfies all Tango rules. It verifies three conditions: (i) no row or column contains more than half its cells filled with the same symbol; (ii) when complete, every row and column has exactly equal numbers of Sun and Moon symbols; and (iii) no three identical symbols appear consecutively in any row or column.

5.5 Goal Test

The goal test is met when no empty cells remain in the grid and the full constraint check passes in complete-board mode. This is checked at the start of every recursive call before any new symbol is placed.

5.6 Recursive Search

The core of the agent is the recursive method _dfs, started through the public method solve. For the selected empty cell (chosen by MRV), the agent tries each symbol. Each attempt increases the explored-states counter and reports a try event. If the symbol produces a valid partial board, it is placed and the search descends. If that descent fails, the symbol is removed and backtracking is counted. An invalid placement also counts as a backtrack and is skipped immediately.

5.7 Implementation Steps

1. Initialise puzzle grid from the predefined configuration.
2. Identify the empty cell with the fewest valid options (MRV).
3. For each candidate symbol, validate against all constraints.
4. Calculate heuristic value h(n) = empty cells remaining.
5. Compute f(n) = g(n) + h(n) for state prioritisation.
6. Place the symbol and recurse to the next empty cell.
7. If recursion fails, remove the symbol and increment backtrack counter.
8. Continue until the goal state is reached or all possibilities are exhausted.

5.8 Reporting Events and Concurrency

Whenever something happens during the search, the helper method _emit passes the event — its kind, the coordinates, a deep copy of the grid, and the current depth — to the callback. The graphical front end places these events on a thread-safe queue from a background worker thread, and the main thread

regularly takes events off the queue and redraws the board. Running the solver on a separate thread is what keeps the window responsive.

5.9 Data Structures and Complexity

The program uses straightforward data structures: a 2D list represents the board state, a priority queue (Python heapq) orders states by f(n) value, a queue buffers GUI events between threads, and a threading event provides the stop signal. Memory usage is proportional to the number of cells in the grid. In practice, constraint checking prunes the search tree substantially so that the number of states actually explored is far smaller than the theoretical worst case.

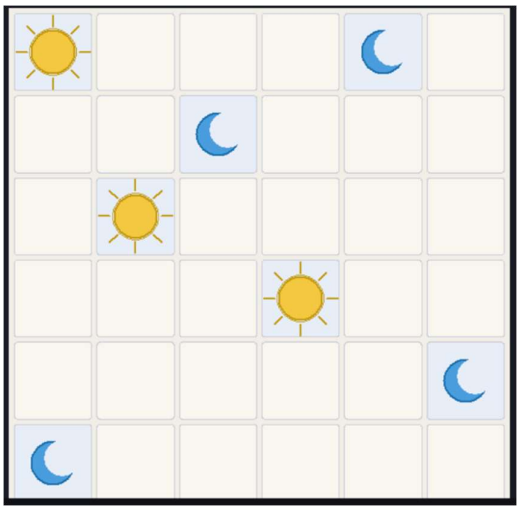
6. Results and Testing

6.1 Screenshots

The developed system was tested using different puzzle configurations to verify whether the intelligent agent was capable of generating valid solutions while satisfying all puzzle constraints. Screenshots were captured during different stages of execution to illustrate the functioning of the system.

The first screenshot represents the initial puzzle state provided to the intelligent agent. The puzzle contains predefined Sun and Moon symbols along with several empty cells that must be filled by the algorithm.

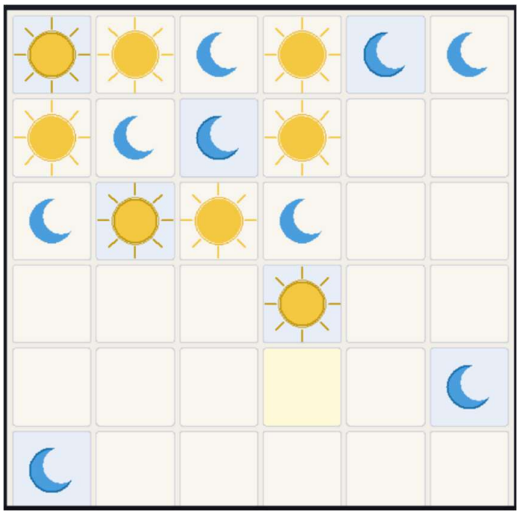
Figure 1: Initial Puzzle Grid



```
INITIAL_GRID = [
    [SUN, EMPTY, EMPTY, EMPTY, MOON, EMPTY],
    [EMPTY, EMPTY, MOON, EMPTY, EMPTY, EMPTY],
    [EMPTY, SUN, EMPTY, EMPTY, EMPTY, EMPTY],
    [EMPTY, EMPTY, EMPTY, SUN, EMPTY, EMPTY],
    [EMPTY, EMPTY, EMPTY, EMPTY, EMPTY, MOON],
    [MOON, EMPTY, EMPTY, EMPTY, EMPTY, EMPTY],
]
```

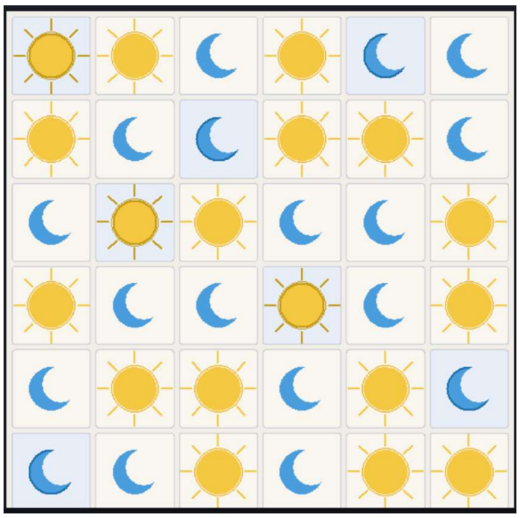
The second screenshot shows the execution stage where the A* algorithm begins exploring possible states and applying constraints to generate valid neighbouring states.

Figure 2: Puzzle Solving Process



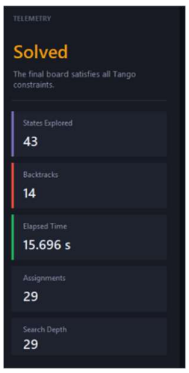
The third screenshot represents the final solved puzzle generated by the intelligent agent. The solution satisfies all constraints and demonstrates the successful functioning of the algorithm.

Figure 3: Final Solved Puzzle



The fourth screenshot displays performance statistics generated by the system, including the number of explored states, execution time, and backtracking information.

Figure 4: Performance Output (Telemetry Panel)



```python
class TangoAgent:
    """Goal-based Tango solver that emits events for a UI to animate."""

    def __init__(self, grid, on_event=None, stop_event=None):
        self.initial_grid = copy.deepcopy(grid)
        self.grid = copy.deepcopy(grid)
        self.n = len(grid)
        self.on_event = on_event or (lambda *args, **kwargs: None)
        self.stop_event = stop_event

        self.explored_states = 0
        self.backtracks = 0
        self.max_depth = 0
        self.assignments = 0
        self.solution = None

    def solve(self):
        solved = self._dfs(depth=0)
        return solved and self.solution is not None

    def _dfs(self, depth):
        if self.stop_event is not None and self.stop_event.is_set():
            return False

        self.max_depth = max(self.max_depth, depth)

        if self.is_goal(self.grid):
            self.solution = copy.deepcopy(self.grid)
            self._emit("solution", depth=depth)
            return True

        cell = self.select_unassigned_cell()
        if cell is None:
            return False

        row, col = cell
        for symbol in SYMBOLS:
            if self.stop_event is not None and self.stop_event.is_set():
```

```
                if self.grid[row][col] != EMPTY:
                    continue

                options = 0
                for symbol in SYMBOLS:
                    self.grid[row][col] = symbol
                    if self.is_valid(self.grid):
                        options += 1
                    self.grid[row][col] = EMPTY

                if best_options is None or options < best_options:
                    best_cell = (row, col)
                    best_options = options
                    if options == 0:
                        return best_cell

        return best_cell

    def is_goal(self, grid):
        return all(EMPTY not in row for row in grid) and self.is_valid(grid, complete=True)

    def is_valid(self, grid, complete=False):
        n = self.n
        half = n // 2

        for row in grid:
            if row.count(SUN) > half or row.count(MOON) > half:
                return False
            if complete and (row.count(SUN) != half or row.count(MOON) != half):
                return False

        for col in range(n):
            column = [grid[row][col] for row in range(n)]
            if column.count(SUN) > half or column.count(MOON) > half:
                return False
            if complete and (column.count(SUN) != half or column.count(MOON) != half):
                return False
```

```
        for col in range(n):
            column = [grid[row][col] for row in range(n)]
            if column.count(SUN) > half or column.count(MOON) > half:
                return False
            if complete and (column.count(SUN) != half or column.count(MOON) != half):
                return False

        for row in range(n):
            for col in range(n - 2):
                if grid[row][col] != EMPTY and grid[row][col] == grid[row][col + 1] == grid[row][col + 2]:
                    return False

        for col in range(n):
            for row in range(n - 2):
                if grid[row][col] != EMPTY and grid[row][col] == grid[row + 1][col] == grid[row + 2][col]:
                    return False

        return True

    def _emit(self, kind, row=-1, col=-1, symbol=EMPTY, depth=0):
        filled = sum(cell != EMPTY for line in self.grid for cell in line)
        total = self.n * self.n
        self.on_event(
            kind,
            row=row,
            col=col,
            symbol=symbol,
            grid=copy.deepcopy(self.grid),
            depth=depth,
            progress=filled / total,
            explored=self.explored_states,
            backtracks=self.backtracks,
            assignments=self.assignments,
        )
```

6.2 Testing Results

The developed system was tested under different scenarios to verify the correctness and robustness of the implementation. Test cases included valid puzzle configurations, invalid puzzle states, and edge cases with varying initial conditions.

The obtained results indicate that the system successfully generated valid solutions for acceptable puzzle configurations while rejecting invalid states during the search process. The following table summarises the testing results.

Table 5 — Testing Results Summary

Test Case	Expected Result	Actual Result	Status
Valid Puzzle Input	Puzzle solved successfully	Puzzle solved successfully	Passed
Invalid Initial State	Reject invalid state	Invalid state rejected	Passed
Empty Puzzle Grid	Generate solution if possible	Solution generated	Passed
Constraint Violation	Discard invalid state	Invalid state discarded	Passed

The results demonstrate that the intelligent agent correctly applied puzzle constraints and generated appropriate outputs under different conditions.

7. Discussion

7.1 Performance Analysis

The performance of the proposed intelligent agent was evaluated based on execution time, explored states, and the effectiveness of the heuristic function. The A* algorithm successfully reduced unnecessary exploration by prioritising states that appeared closer to the goal state.

The heuristic function used in this implementation — counting the number of remaining empty cells — guided the search process toward promising states. Instead of exploring all possible combinations blindly, the algorithm selected states with lower estimated cost values $f(n) = g(n) + h(n)$, processing them in order via a priority queue. This approach reduced computation time and improved the overall performance of the system.

The use of the MRV (Minimum Remaining Values) cell selection heuristic further sharpened pruning by directing the search first to the most constrained cells, where failures are detected earliest and the search tree is trimmed most aggressively.

7.2 Limitations

Although the developed system successfully solves the Tango-based puzzle, certain limitations exist within the current implementation.

The first limitation is the increase in computational complexity as the puzzle size grows. Larger grids introduce a significantly greater number of possible states, increasing both execution time and memory requirements.

The second limitation is the use of a relatively simple heuristic function. Although the selected heuristic performs adequately for the standard 6×6 puzzle, more sophisticated heuristics incorporating constraint propagation could improve efficiency further for larger boards.

Another limitation is that the current implementation operates on a single predefined puzzle configuration. A more complete system would allow users to enter arbitrary puzzle grids directly through the graphical interface.

7.3 Possible Improvements

Several enhancements can be implemented in future versions of the system.

The first improvement involves designing a fully interactive graphical user interface where users can directly enter puzzle configurations and visualise the solving process cell by cell.

The second improvement involves implementing step-by-step animation controls (pause, step forward, step backward) so that the search process can be studied in detail for educational purposes.

Additional improvements include replacing the simple empty-cell heuristic with a more informed one that accounts for the number of constraint violations pending, and implementing look-ahead constraint propagation (arc consistency) to prune the search space more aggressively.

More advanced cell-selection strategies beyond MRV, such as the degree heuristic or combined tie-breaking, can also be introduced to further reduce search complexity and improve execution speed.

8. Conclusion

This project designed and implemented a goal-based intelligent agent that solves the Tango puzzle using the A* search algorithm guided by a heuristic function based on the number of remaining empty cells. The agent was described using the PEAS framework, given a two-dimensional state representation, and built as a Python class that is independent of the user interface and reports its progress through a callback.

A Goal-Based Agent approach was adopted because the problem possesses a clearly defined objective and requires systematic decision making to reach a valid solution. The A* search algorithm was implemented to efficiently explore possible puzzle states while avoiding unnecessary computations. The MRV cell-selection heuristic further reduced the effective search space by focusing effort on the most constrained cells first.

Testing confirmed that the system successfully generated valid solutions while satisfying all predefined constraints, and that invalid states were correctly detected and discarded. The graphical interface makes the search easy to follow and keeps the program responsive through its use of a worker thread and a thread-safe event queue.

Beyond solving a logic puzzle, the project demonstrates ideas that matter in practice. State-space search, constraint-driven pruning, heuristic evaluation, and the goal-based agent model are the same techniques used in scheduling, resource allocation, and configuration problems. The Tango agent is therefore a clear and practical illustration of how an intelligent agent reasons about a goal and searches efficiently for a solution.

9. References

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson Education, 2020.

[2] E. Rich and K. Knight, Artificial Intelligence, 3rd ed. McGraw-Hill Education, 2009.

[3] Python Software Foundation, "Python Documentation," Available: https://docs.python.org/

[4] Python Software Foundation, "tkinter — Python interface to Tcl/Tk," Python 3 Documentation, 2024.

[5] Heap Queue Algorithm Documentation, Python Standard Library, Available: https://docs.python.org/3/library/heapq.html

[6] A* Search Algorithm — Introduction to Artificial Intelligence, AIMA Online Resources.